Autonomous SG 2026: Challenge #2 Game Plan

Oct 31, 2026 · @Rohit

We build three separate agents (ATC, Pilot, Ground) that negotiate a disrupted arrival from one real 2015 DOT delay row. Deterministic code checks every safety constraint, caps the talk at 3 turns, and decides when there's no consensus. No Killswitch tie-in. Submit by 15:30.

The core design decision

The LLMs negotiate trade-offs; they never decide what is safe and never decide when to stop. That one rule answers all three "what to avoid" bullets and the rubric's technical soundness criterion (25%), which marks down "an LLM doing scheduling or legality checks".

Job	Who does it	Why
Turn the DOT row into a disruption event	Deterministic parser	Same input, same event, every time
Generate candidate resolutions (arrival slot, gate, hold or divert)	Deterministic enumerator	Agents choose and argue; they don't invent options that break physics
Rank candidates against each domain's goals, propose, accept, reject, counter-offer with reasons	3 separate LLM agents, each with only its own domain's data	This is the negotiation the statement asks for
Wake separation, fuel reserve, gate clash, crew hours	Deterministic validators per domain	A validator veto overrides any LLM "accept"
Turn limit and convergence	Deterministic coordinator, hard cap of 3 rounds	No endless loops by construction
Decide when there's no consensus	Deterministic fallback (lowest weighted cost among feasible candidates)	The non-AI fallback the statement requires

Job	Who does it	Why
Final approval	Human duty manager, one click	The rubric's "human decides"; agents act without humans *until* consensus, as the statement says

The coordinator is code, not a fourth LLM. A fourth LLM as mediator would just be the "single prompt" problem in another place.

Fallback ladder. The LLM negotiation is the non-deterministic primary path; three deterministic levels sit under it. The UI and the log always say which level produced the plan.

Level	Path	Trigger	Deterministic?
L0	LLM agents negotiate	Default	No (model, temperature and prompt hash are logged)
L1	A rule-based policy stands in for one agent for that round	Timeout, invalid JSON after 1 retry, or picks an illegal option	Yes
L2	The coordinator picks the lowest-cost option every validator passes	No consensus after round 3, or the API is down	Yes
L3	Escalate to the duty manager with "recommend divert"	No option passes the validators	Yes

`--offline` runs every agent at L1 with no API calls. It's the deterministic baseline, and the stage backup if the Wi-Fi dies.

Data gaps and organiser questions

The DOT row gives us a flight, its delay and its cause, and nothing the agents actually negotiate over. It has no runway, gate, fuel on board, aircraft wake category, passenger connections or crew shift. We have to generate that state, and we must label it synthetic on screen.

What the row does give us [Likely, from the public 2015 dataset]: date, airline, flight and tail number, origin and destination, scheduled and actual times, taxi times, distance, arrival delay, and the delay causes (`WEATHER_DELAY`, `LATE_AIRCRAFT_DELAY`,

`AIR_SYSTEM_DELAY` and so on). The `airlines.csv` and `airports.csv` you attached are from that same dataset, so you already have 2 of its 3 files.

Data traps:

- `flights.csv` is about 5.8 million rows [Likely]. Load only the columns we need, in chunks, and filter once to a small `scenarios.csv`.
- In October 2015 rows, the airport columns hold 5-digit numeric IDs instead of IATA codes [Likely]. Skip October.
- Use the destination airport's coordinates from `airports.csv` for the map and the arrival agent.

Ask the organisers before 11:30:

1. **May we generate the operational state (runway config, gates, fuel, connections, crew) from the row, labelled synthetic?** Default if no answer: yes, seeded from the row so it's reproducible.
2. **Does "under 3 turns" mean 3 rounds of all three agents, or 3 messages in total?** Default: 3 rounds, hard cap.
3. **One showcase row, or should we show it working across many rows?** Default: one showcase row live, plus a batch run over 50+ rows for evaluation.

What we build

One Python package, three agent calls per round, one coordinator, one log. Everything an agent says is a JSON message that passes a schema before anyone else sees it.

1. **Scenario builder.** Take a row with `WEATHER_DELAY > 15` or `LATE_AIRCRAFT_DELAY > 20`. Seed a random generator with the row's ID and generate the airport state: active runway, arrival queue with wake categories, gates with occupancy, fuel on board above reserve, onward connections and crew shift end. The same row always gives the same state.
2. **Candidate enumerator.** Arrival slots every 2 minutes over the next 60, × free gates, plus "hold" and "divert to the nearest alternate". Typically a few hundred options.
3. **Validators (code).** Each candidate gets pass or fail per domain, with a reason:
   - ATC: wake separation behind the preceding arrival (3–6 NM by category, converted to seconds at approach speed) and the runway's hourly arrival cap [Likely, check against FAA 7110.65].
   - Pilot: fuel at landing ≥ 45 minutes reserve plus alternate [Likely, FAA Part 121 domestic]; holding burn computed per minute.
   - Ground: no gate overlap within a 10-minute buffer; ramp crew finishes before shift end or the overtime is costed.

4. **Agents (3 separate Haiku calls).** Each sees the event, the feasible candidates, and only its own domain's state. Each returns `{top_choices, scores, reasons, red_lines}`. The Pilot agent never sees gate data; the Ground agent never sees fuel.
5. **Coordinator protocol, 3 rounds maximum.**
   a. Round 1: each agent ranks the candidates and states its red lines.
   b. Round 2: the coordinator proposes the candidate with the best combined rank that every validator passes. Each agent accepts or rejects with a reason code and may name one counter-offer.
   c. Round 3: if all three accept, that's consensus. If not, the coordinator offers the best counter-offer; failing that, the deterministic fallback picks and the log says "fallback".
6. **Consensus log.** Append-only JSONL: message ID, round, sender, payload, validator results, and the hash of the previous line. Any edit to the log breaks the chain, which makes it tamper-evident.
7. **UI (Streamlit).** Row picker, including a "random row" button for judges. Three agent columns, round by round. A constraint table in green and red. The final plan with a duty-manager Approve button. The log, downloadable.

The message every agent returns:

```
{"agent":"PILOT","round":2,"decision":"REJECT","candidate_id":"S14-G7","reason_code":"FUEL_MARGIN_THIN","reason":"14 min hold is legal but leaves 52 min reserve vs 45 min minimum; 3 passengers miss a connection","counter":"S08-G7"}
```

Agents only reject on preference among legal options. An option that breaks a red line never reaches them, because the validators filter it first.

Where the points are

Polish isn't scored. Each of the three judges asks "where does this fail, and how would your user know?" Build for that question.

Criterion	Weight	What we show
Technical soundness	25%	3 independent agent calls, code validators, a 3-round cap, a non-AI fallback. A judge picks a random row live and it runs end to end

Criterion	Weight	What we show
Problem realism	20%	Named user: the airline's station duty manager at the arrival hub, coordinating with approach control and ramp. Status quo: phone calls between ATC flow, the operations centre and ramp control while the aircraft holds. One real row, e.g. an ATL arrival with a weather delay
Honest evaluation	20%	Batch run over 50+ rows: consensus rate, rounds used, vetoes, fallback rate, vs a first-come-first-served baseline. Synthetic state labelled
Trust and traceability	20%	Hash-chained consensus log; every rejection has a reason code; validator results visible per candidate; the duty manager approves
Impact	10%	Modelled delay minutes, holding fuel and missed connections vs the baseline, labelled MODELLED, not measured
Time discipline	5%	Submit by 15:30, pitch under 3:00

Timeline

About 4 hours of build time are left, so every slot is a gate: miss it and we cut scope, not time. Features freeze at 15:00; submit at 15:30.

Time	Rohit (coordinator and agents, M1 Pro)	Aadith (data, validators, evaluation)	Thejus (UI, pitch)
11:15–11:30	Fresh repo, `.env` gitignored, `anthropic` + `pydantic` installed, key with a spend limit	Download `flights.csv`; filter to `scenarios.csv` (qualifying rows, no October)	Ask the organisers the three questions; write the answers in this doc
11:30–12:30	Message schemas; coordinator loop with **stub agents** that return fixed JSON; hash-chained log	Scenario builder (seeded state); candidate enumerator; the three validators with unit tests	Streamlit skeleton: row picker, three agent columns, log panel, reading a saved `run.json`

Time	Rohit (coordinator and agents, M1 Pro)	Aadith (data, validators, evaluation)	Thejus (UI, pitch)
12:30 gate	One row runs end to end with stub agents and produces a log. No: cut the counter-offer step		
12:30–13:45	Replace stubs with three Haiku agents; prompts; schema validation with one retry, then fallback	Deterministic fallback and FCFS baseline; batch runner over N rows	Wire the UI to live runs; constraint table in green and red; Approve button
13:45–14:15	All: lunch, then record a backup screen video of one full negotiation		
14:15–15:00	Adversarial tests: a prompt that tells the Pilot agent to accept an illegal hold; malformed JSON; a row with no feasible slot	Batch run over 50+ rows; metrics table; pick the failure case to show	Slides: 5 at most, built from real outputs
15:00 freeze	Fixes only		
15:00–15:25	All: rehearse the 3-minute pitch twice with a timer		
15:30	Submit		
16:30	Demo		

Cut order if we slip: counter-offers first (agents just accept or reject), then the random-row button, then batch size drops to 20 rows. The separate agents, the validators, the turn cap, the fallback and the log never get cut; they are the brief.

Honest evaluation pack

One slide with the batch numbers, one with a failure. Most teams will show one perfect scripted negotiation; we show the system over many rows and where it breaks.

Batch run over 50+ qualifying rows, same seeds for every method:

Metric	Negotiated	FCFS baseline	Fallback only
Consensus within 3 rounds (%)		n/a	n/a
Mean rounds used		n/a	n/a
Unsafe plans accepted (must be 0)			
LLM "accepts" vetoed by validators		n/a	n/a
Modelled delay (min), holding fuel (kg), missed connections			
Cost per negotiation (US$) and latency (s)		0	0

Also report the single-prompt ablation if there's time: one LLM playing all three roles on the same rows. If it accepts more unsafe plans or ignores a domain, that's the evidence for why the roles stay separate.

Failure cases to show on purpose:

1. **An injected unsafe accept.** Tell the Pilot agent to accept a 25-minute hold. The fuel validator vetoes it and the log shows the veto. This is the most persuasive 20 seconds of the demo.
2. **No consensus.** A row where Ground and Pilot can't agree within 3 rounds. The fallback picks, and the UI says "fallback, not consensus".
3. **The synthetic state.** Gates, fuel and connections are generated, not real. Real results need airline and airport feeds; say so on the slide.
4. **What the model leaves out.** One flight at a time; it doesn't re-sequence the whole arrival bank or model the knock-on to the aircraft's next leg.

The 3-minute pitch

The demo carries it; slides only frame it. Thejus times every rehearsal.

1. **0:00–0:25 User and pain.** The station duty manager at a hub, a real 2015 flight with a weather delay, and three parties on the phone while the aircraft holds and burns fuel.
2. **0:25–1:40 Live demo.** A judge picks a row. The three agents negotiate round by round, the constraint table turns green, consensus arrives, and the duty manager approves.
3. **1:40–2:15 Safety.** The injected unsafe accept, vetoed by the fuel validator, with the log line on screen.
4. **2:15–2:45 Evidence.** The batch table: consensus rate, zero unsafe plans, the fallback rate, modelled delay vs FCFS.

5. **2:45–3:00 Limits and next step.** Synthetic state, one flight at a time. Next: real gate and fuel feeds.

Never say "no human needed" or "autonomous ATC". Say "agents negotiate, code enforces safety, a person approves".

Credits and stack

The $50 runs out on coding, not on the agents. One negotiation is at most 9 Haiku calls (3 agents × 3 rounds). A 50-row batch is a few hundred calls, likely a few dollars [Guessing; check current Haiku pricing in the console].

- Set a spend limit on the key before anything else.
- Agents use the current Haiku. Coding help uses Sonnet; avoid the top model for routine code.
- Cache each agent's system prompt; send only the candidate list and the messages, not the whole scenario each round.
- Run the batch once, save the results, and demo from live single runs. Don't re-run the batch on stage.
- Keys live in a gitignored `.env`, never in slides or commits.

Stack: Python 3, `pandas`, `pydantic` (message schemas), `anthropic`, `streamlit`, `pytest` for the validators. Install it all in the first 15 minutes.

- [ ] Organiser answers on synthetic state, the meaning of "3 turns", and showcase vs batch written in this doc
- [ ] `flights.csv` downloaded and filtered to `scenarios.csv`
- [ ] Confirm the submission time and format with the organisers